

07-medallion-approach

Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Share

Publish

Cmd 1

Python

```
class Bronze():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def getSchema(self):
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
        CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
        PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
        DeliveryType string,
        DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,
        State string>,
        InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
        ItemPrice double, ItemQty bigint, TotalValue double>>
        """

    def readInvoices(self):
        return (spark.readStream
                .format("json")
                .schema(self.getSchema())
                .load(f"{self.base_data_dir}/data/invoices")
                )

    def explodeInvoices(self, invoiceDF):
        return (invoiceDF.selectExpr("InvoiceNumber", "CreatedTime", "StoreID", "PosID",
```



Python ▾

▶ Run all

- Connect ▾

```
def readInvoices(self):
    return (spark.readStream
            .format("json")
            .schema(self.getSchema())
            #.option("cleanSource", "delete")
            .option("cleanSource", "archive")
            .option("sourceArchiveDir", f"{self.base_data_dir}/data/invoices_archive")
            .load(f"{self.base_data_dir}/data/invoices")
    )

def process(self):
    print(f"\nStarting Bronze Stream...", end='')
    invoicesDF = self.readInvoices()
    sQuery = (invoicesDF.writeStream
              .queryName("bronze-ingestion")
              .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/invoices")
              .outputMode("append")
              .toTable("invoices_bz")
    )
    print("Done")
```

Cmd 2

Python

```
class Silver():  
    def __init__(self):  
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"  
  
    def explodeInvoices(self, invoiceDF):  
        return ( invoiceDF.selectExpr("InvoiceNumber", "CreatedTime", "StoreID", "PosID",  
                                       "CustomerType", "PaymentMethod", "DeliveryType", "DeliveryAddress.City",  
                                       "DeliveryAddress.State", "DeliveryAddress.PinCode",  
                                       "explode(InvoiceLineItems) as LineItem")  
                )  
  
    def flattenInvoices(self, explodedDF):  
        from pyspark.sql.functions import expr  
        return( explodedDF.withColumn("ItemCode", expr("LineItem.ItemCode"))  
                .withColumn("ItemDescription", expr("LineItem.ItemDescription"))  
                .withColumn("ItemPrice", expr("LineItem.ItemPrice"))  
                .withColumn("ItemQty", expr("LineItem.ItemQty"))  
                .withColumn("TotalValue", expr("LineItem.TotalValue"))  
                .drop("LineItem")  
                )  
  
    def appendInvoices(self, flattenedDF):
```



07-medallion-approach

Python ▾

File Edit View Run Help [Last edit was 18 minutes ago](#) [Provide feedback](#)

▶ Run all

● Connect ▾

Share

Publish

Cmd 2

Python ▶ ▾ - x

```
class Silver():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def readInvoices(self):
        return ( spark.readStream
                .table("invoices_bz")
                )

    def explodeInvoices(self, invoiceDF):
        return ( invoiceDF.selectExpr("InvoiceNumber", "CreatedTime", "StoreID", "PosID",
                                       "CustomerType", "PaymentMethod", "DeliveryType", "DeliveryAddress.City",
                                       "DeliveryAddress.State", "DeliveryAddress.PinCode",
                                       "explode(InvoiceLineItems) as LineItem")
                )

    def flattenInvoices(self, explodedDF):
        from pyspark.sql.functions import expr
        return( explodedDF.withColumn("ItemCode", expr("LineItem.ItemCode"))
                .withColumn("ItemDescription", expr("LineItem.ItemDescription"))
                .withColumn("ItemPrice", expr("LineItem.ItemPrice"))
                .withColumn("ItemQty", expr("LineItem.ItemQty"))
```



```
        return( explodedDF.withColumn("ItemCode", expr("LineItem.ItemCode"))
                .withColumn("ItemDescription", expr("LineItem.ItemDescription"))
                .withColumn("ItemPrice", expr("LineItem.ItemPrice"))
                .withColumn("ItemQty", expr("LineItem.ItemQty"))
                .withColumn("TotalValue", expr("LineItem.TotalValue"))
                .drop("LineItem")
            )

    def appendInvoices(self, flattenedDF):
        return (flattenedDF.writeStream
                .queryName("silver-processing")
                .format("delta")
                .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/invoice_line_items")
                .outputMode("append")
                .toTable("invoice_line_items")
            )

    def process(self):
        print(f"Starting Invoice Processing Stream...", end='')
        invoicesDF = self.readInvoices()
        explodedDF = self.explodeInvoices(invoicesDF)
        resultDF = self.flattenInvoices(explodedDF)
        sQuery = self.appendInvoices(resultDF)
        print("Done\n")
        return sQuery
```



07-medallion-approach

Python ▾

File Edit View Run Help

Last edit was 21 minutes ago

Provide feedback

▶ Run all

● Connect ▾

Run

Publish



```
.format("delta")
.option("checkpointLocation", f"{self.base_data_dir}/checkpoint/invoice_line_items")
.outputMode("append")
.toTable("invoice_line_items")
)
```

```
def process(self):
    print(f"\nStarting Silver Stream...", end='')
    invoicesDF = self.readInvoices()
    explodedDF = self.explodeInvoices(invoicesDF)
    resultDF = self.flattenInvoices(explodedDF)
    sQuery = self.appendInvoices(resultDF)
    print("Done\n")
    return sQuery
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



Cmd 1

%run ./07-medallion-approach

Cmd 2

```
class medallionApproachTestSuite():  
    def __init__(self):  
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"  
  
    def cleanTests(self):  
        print(f"Starting Cleanup...", end='')  
        spark.sql("drop table if exists invoice_bz")  
        spark.sql("drop table if exists invoice_line_items")  
        dbutils.fs.rm("/user/hive/warehouse/invoice_bz", True)  
        dbutils.fs.rm("/user/hive/warehouse/invoice_line_items", True)  
  
        dbutils.fs.rm(f"{self.base_data_dir}/checkpoint/invoices_bz", True)  
        dbutils.fs.rm(f"{self.base_data_dir}/checkpoint/invoice_line_items", True)  
  
        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices_archive", True)  
        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices", True)  
        dbutils.fs.mkdirs(f"{self.base_data_dir}/data/invoices")  
        print("Done")
```



```
dbutils.fs.mkdirs(rf"{self.base_data_dir}/data/invoices")
print("Done")

def ingestData(self, itr):
    print(f"\tStarting Ingestion...", end='')
    dbutils.fs.cp(f"{self.base_data_dir}/datasets/invoices/invoices_{itr}.json", f"{self.base_data_dir}/data/
invoices/")
    print("Done")

def assertResult(self, expected_count):
    print(f"\tStarting validation...", end='')
    actual_count = spark.sql("select count(*) from invoice_line_items").collect()[0][0]
    assert expected_count == actual_count, f"Test failed! actual count is {actual_count}"
    print("Done")

def waitForMicroBatch(self, sleep=30):
    import time
    print(f"\tWaiting for {sleep} seconds...", end='')
    time.sleep(sleep)
    print("Done.")

def runTests(self):
    self.cleanTests()
    bzStream = Bronze()
    bzQuery = bzStream.process()
```




```
print(f"\tWaiting for {sleep} seconds...", end='')  
time.sleep(sleep)  
print("Done.")
```

```
def runTests(self):  
    self.cleanTests()  
    bzStream = Bronze()  
    bzQuery = bzStream.process()  
  
    slStream = Silver()  
    slQuery = slStream.process()  
  
    print("\nTesting first iteration of invoice stream...")  
    self.ingestData(1)  
    self.waitForMicroBatch()  
    self.assertResult(1249)  
    print("Validation passed.\n")  
  
    print("Testing second iteration of invoice stream...")  
    self.ingestData(2)  
    self.waitForMicroBatch()  
    self.assertResult(2506)  
    print("Validation passed.\n")  
  
    print("Testing third iteration of invoice stream...")  
    self.ingestData(3)
```



```
print("Validation passed.\n")

print("Testing third iteration of invoice stream...")
self.ingestData(3)
self.waitForMicroBatch()
self.assertResult(3990)
print("Validation passed.\n")

bzQuery.stop()
slQuery.stop()

print("Validating Archive...", end="")
archives_expected = ["invoices_1.json", "invoices_2.json"]
for f in dbutils.fs.ls(f"{self.base_data_dir}/data/invoices_archive/{self.base_data_dir}/data/invoices"):
    assert f.name in expected, f"Archive Validation failed for {f.name}"
print("Done")
```

Cmd 3

```
maTS = medallionApproachTestSuite()
maTS.runTests()
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



08-medallion-approach-test-suite Python

File Edit View Run Help Last edit was 10 minutes ago Provide feedback

Run all

Connect

Share

Publish

```
print("Validation passed.\n")

print("Testing third iteration of invoice stream...")
self.ingestData(3)
self.waitForMicroBatch()
self.assertResult(3990)
print("Validation passed.\n")

bzQuery.stop()
slQuery.stop()

print("Validating Archive...", end="")
archives_expected = ["invoices_1.json", "invoices_2.json"]
for f in dbutils.fs.ls(f"{self.base_data_dir}/data/invoices_archive/{self.base_data_dir}/data/invoices"):
    assert f.name in archives_expected, f"Archive Validation failed for {f.name}"
print("Done")
```

Cmd 3

```
maTS = medallionApproachTestSuite()
maTS.runTests()
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



Hostname: ec2-54-188-18-175.us-west-2.compute.amazonaws.com · Spark Version: 12.2.x-scala2.12

[Open in new tab](#)[Jobs](#) [Stages](#) [Storage](#) [Environment](#) [Executors](#) [SQL / DataFrame](#) [JDBC/ODBC Server](#) [Structured Streaming](#)

Streaming Query

▼ Active Streaming Queries (2)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Name	Status	ID	Run ID	Start Time ▾	Duration	Avg Input /sec	Avg Process /sec	Latest Batch
silver-processing	RUNNING	9dbff29-c19d-46e9-9259-be5ab1be7727	8a9aceda-2a04-4e7b-b04c-5251112217b0	2023/10/18 16:43:16	55 seconds 565 ms	538.61	24.93	1
bronze-ingestion	RUNNING	3b5b2c6d-7bc0-46b7-923c-f033ad3ed9ab	1b2ab43e-f23d-43d5-a596-6743cf362275	2023/10/18 16:43:03	1 minute 8 seconds	271.05	18.05	1

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go



Hostname: ec2-54-188-18-175.us-west-2.compute.amazonaws.com · Spark Version: 12.2.x-scala2.12

[Open in new tab](#)[Jobs](#) [Stages](#) [Storage](#) [Environment](#) [Executors](#) [SQL / DataFrame](#) [JDBC/ODBC Server](#) [Structured Streaming](#)

Streaming Query

▼ Completed Streaming Queries (2)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Name	Status	ID	Run ID	Start Time ▾	Duration	Avg Input /sec	Avg Process /sec	Latest Batch	Error
silver-processing	FINISHED	9dbfff29-c19d-46e9-9259-be5ab1be7727	8a9aceda-2a04-4e7b-b04c-5251112217b0	2023/10/18 16:43:16	1 minute 43 seconds	324.08	19.03	3	-
bronze-ingestion	FINISHED	3b5b2c6d-7bc0-46b7-923c-f033ad3ed9ab	1b2ab43e-f23d-43d5-a596-6743cf362275	2023/10/18 16:43:03	1 minute 53 seconds	233.53	17.83	3	-

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

